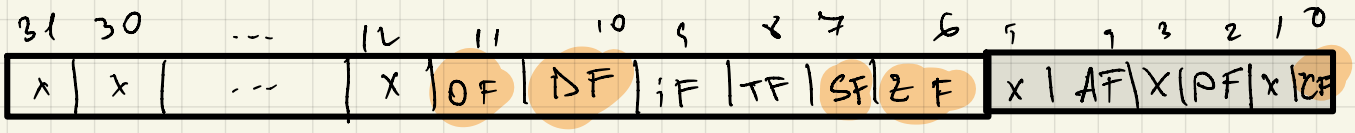
6 segment registers (16 bits) FLAGS reg 32 bits, but use only 9

You need to know all 9 flags for exam (marked w/ yellow)



* Why they didn't implement odd w instruction like they did w. multiplication and division.

- Because the overflow can only be 1 (or 0)
- it is useless to assign 1 byte for it (8 times its size)
- operations (like addition) are incorrect mathematically, that's why the carry flag shows you this. so that you can fix it.

We are forgetting a conversion from a byte to a word, using cdq, etc

* Why the designer of the CPU, provided two diff. flags for overflow?

- Bc. we have 2 possible interpretations in base 2. (and they have diff. values)

* How do you tell the CPU to perform an addition in the **unsigned interpretation**?

- You cannot do that. Bc that add. will have **two possible interpretations** (signed / unsigned) (OF, CF)

- for mul & div. you have to decide before the case (signed / unsigned)

mul → imul
div → idiv.

* Why we **don't** have iadd, isub.

Bc. the addition goes in the same way.

PF - used in the transmission of data.
(for each byte sent.)

How many bits w/ the value 1 we have?

FOR.

```
1001    0011  +
0111    0011
-----------
0000    0110
```

Parity Flag: PF ⇒ 1 (result is even)

Auxiliary: AF ⇒ 0 ← (carry flag for nibbles) (4 BITS)
(shows the transport val. from bit 3 to bit 4)

Carry [CF] => [1] (FLAGS THE UNSIGNED OVERFLOW)

— — — — — — — VERY IMPORTANT ↓ — — — — —

Zero [ZF] => [0] (1 if the LPO was 0, or 0 otherwise)

Sign [SF] => [0] (1 if the LPO < 0, or 0 otherwise)

Trap [TF] => [1] (debugging flag => 1 => machine stops after every instruction)

Interrupt [IF] => [0] (interrup flag (CAN'T USE ON 32 BITS ONLY ON 16 BITS)

1 => interrupts are allowed, 0 otherwise.

(used for viruses, they redirect the interrupts)

"cli" => clear interrupt.

(clears the flag)

CRITICAL SECTION

```
mov eax, ...
... ← CRITICAL SECTION
add ebx, ecx
```

- no one must have the power to interrupt that.

"sti" => set interrupt

(turns it back to 1)

=> NOT working on 32 bits.

Direction [DF] => [0]

(not set by the LPO.)

(supposed to be set by the programmer.)

(0 => string parsing ascending

1 => — " — descending)

- flags that show us what happened
  - CF, OF, PF, ZF, AF, SF
- flags that have to be set by the dev. for influencing the execution of some instruction.
  - TF, IF, DF, CF

DF => { cld (DF = 0) "clear direction flag"
      { std (DF = 1) "set" -"-

CF => { clc (CF = 0) "clear carry flag"
      { stc (CF = 1) "set" -"-
      { cmc (CF = !CF) "complement carry flag"

IF => { cli (IF = 0)
      { sti (IF = 1) -"-

Address of a memory location = nr. of consecutive bytes from the beginning of the RAM memory

1 MB = 2^{20} (20 bits NOT FIT ON 16)

[0]	[7]
segm	offset
16 bits	16 bits

Nr.	Starting Address - 32 bits	Limit.
3	0752 AB h	16 GB
8	7CD 90 B h	64GB
.	.	4GB
.	.	
.	.	

← Limit (size of that segment)

Invalid Address Specification
Memory Violation Error — Runtime error thrown by the OS if it does not find the address when you call sgm:off. or a[7]

- You don't know those addresses. Only the OS knows.

⇒ a block of discrete size called a physical segment is.

$2^{16} = \underbrace{2^{6}}_{64} \cdot \underbrace{2^{10}}_{1KB} = 64\,KB \Rightarrow 16\ BITS$

$2^{32} = \underbrace{2^{2}}_{4} \cdot \underbrace{2^{30}}_{1GB} = 4\,GB \Rightarrow 32\ BITS$

- a variable-sized block of memory called a logical segment occupied by a program's code or data

- Why all the other registers were extended to 32 bits, but not CS, DS, SS, ES, FS, GS? (the segment registers)

- BC. these registers in 16 bits were holding the starting addresses of those segments, but in 32 bits they are not allowed to hold anymore these starting addresses. In 32 BITS they are only holding a so called **segment descriptor** (an index in a table managed entirely by the OS)

=> We don't need 32 bits values to express an index in a table

Eg: address specification => 8:1000h

$a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0 := 0000000 0 + 0_7 0_6 0_5 0_4 0_3 0_2 0_1 0_0$

↓

$a_7 0_6 a_5 a_4 a_3 a_2 a_1 a_0 := 0_7 0_6 0_5 0_4 0_3 0_2 0_1 0_0$

=> flat memory model

- real mode on 16 bits (memory of word 16 bits : 4MB memory)
- protected mode on 32 bits (NOW 32 bits ; 4GB memory)

Every program is composed by one or more segments:

- code segment CS
- data segment DS
- stack segment SS
- extra segment ES

At most ONE of these segments active at one given moment.

* What does the CS contain?

Contain the segment selector corresponding to the active code segment.
(same for other 3)

EIP = Extended Instruction Pointer

it contains the offset of the current instruction inside the current code segment, this register being managed by the ALU.

segm : offset	=> FAR address